ECommerce Project — Documentation

> **Version:** 1.4.1
>
> Project for Advance Computer Programming
>
> **Project advisor** : Sir Tayyab Khushi

Team Members

Name	Roll No
Aman Ali (L)	195
Umar	139
Aiza Noor	172
Haida Aleem	162
Javeria Manzoor	201

Table of Contents

---

1. Project Overview

An E-Commerce backend system built as an academic project demonstrating:

- **3 Design Patterns** (Factory, Strategy, Singleton)
- **5 Persistence Formats** (JSON, XML, Pickle, SQLite, GZIP + plain Text)
- **Full OOP** (ABC, Encapsulation, Polymorphism, Dataclass, Enums)
- **REST API** via Flask serving a live single-page dashboard

---

2. Directory Structure

```
ECommerce_Project/
│
├── app.py                  Flask REST API server (entry point for web)
├── main.py                 CLI demo / seed script (entry point for console)
├── requirements.txt        Python dependencies
│
├── models/                 ── Domain layer (WHAT the data is)
│   ├── product.py          Abstract base class + 3 concrete product types
│   ├── user.py             User dataclass + UserRole enum
│   └── order.py            Order class + OrderStatus enum
│
├── patterns/               ── Design Patterns
│   ├── factory.py          ProductFactory  (FACTORY pattern)
│   ├── singleton.py        Logger          (SINGLETON pattern)
│   └── strategy.py         Payment methods (STRATEGY pattern)
│
├── persistence/            ── Storage layer (5 formats)
│   ├── db_handler.py       SQLite  (primary relational DB)
│   ├── json_handler.py     JSON    (human-readable backup)
│   ├── xml_handler.py      XML     (structured markup)
│   ├── pickle_handler.py   Pickle  (binary Python serialization)
│   └── gzip_handler.py     GZIP    (compressed archive)
│
├── utils/
│   ├── helpers.py          buy_product, transfer, calculate_discount,
│   │                       save_transaction (→ data/transactions.txt)
│   └── exceptions.py       Custom exception hierarchy
│
├── tests/
│   └── test_suite.py       pytest unit tests
│
├── templates/
│   └── index.html          Single-page dashboard (Jinja2)
│
├── static/
│   ├── css/style.css       Dashboard styles
│   └── js/main.js          Frontend logic (API calls, modals, charts)
│
└── data/                   ── Auto-created at runtime
    ├── database.db         SQLite database
    ├── products.json       JSON backup
    ├── products.xml        XML backup
    ├── products.pkl        Pickle binary backup
    ├── app.log             Singleton Logger output
    ├── app_logs.gz         Compressed log archive
    └── transactions.txt    Plain-text purchase ledger
```

---

3. Design Patterns

4.1 Factory Pattern — `patterns/factory.py`

Purpose: Create different Product types without exposing instantiation logic.

```
 # Instead of this (bad — tight coupling):
if type == "electronics":
    p = Electronics(id, name, price, stock, warranty_years=2)
elif type == "clothing":
    p = Clothing(id, name, price, stock, size="M")

# We use this (Factory — loose coupling):
product = ProductFactory.create_product("electronics", 101, "Laptop", 95000, 13)
```

Input Type	Output Class	Extra Field
`"electronics"`	`Electronics`	`warranty_years`
`"clothing"`	`Clothing`	`size`
`"grocery"`	`Grocery`	`expiry_date`

3.2 Singleton Pattern — `patterns/singleton.py`

Purpose: Only ONE Logger instance exists across the entire application.

```
class Logger:
    _instance = None          # class-level variable — shared by ALL instances

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            # open log file once here
        return cls._instance   # always returns the SAME object
```

Proof it works:

```
a = Logger()
b = Logger()
print(a is b)  # True — same object!
```

Why Singleton for Logger?

- Prevents multiple log files being opened simultaneously
- All modules write to the same `data/app.log`
- Avoids race conditions in file writing

3.3 Strategy Pattern — `patterns/strategy.py`

Purpose: Swap payment algorithms at runtime without `if/elif` chains.

```
# All payment classes implement the same interface (ABC):
class PaymentStrategy(ABC):
    @abstractmethod
    def pay(self, amount: float) -> bool: ...

class CreditCard(PaymentStrategy):
    def pay(self, amount): ...   # credit card logic

class JazzCash(PaymentStrategy):
    def pay(self, amount): ...   # jazzcash logic

class CashOnDelivery(PaymentStrategy):
    def pay(self, amount): ...   # COD logic
```

At runtime (in `Order.checkout()`):

```
# Order doesn't care WHICH payment — it just calls .pay()
self.payment.pay(self.total)   # polymorphism in action!
```

Strategy Class	Payment Method
`CreditCard`	Card number based
`JazzCash`	Mobile wallet

Strategy Class	Payment Method
`EasyPaisa`	Mobile wallet
`CashOnDelivery`	Physical cash

---

4. Persistence Layer

Why 5 formats?

Becaue Lecture demands and :

Format	File	Why Used	Readable?
JSON	`products.json`	Primary backup, human-readable	✅ Yes
XML	`products.xml`	Used in enterprise web services	✅ Yes
Pickle	`products.pkl`	Full Python object serialization	❌ Binary
SQLite	`database.db`	Relational DB, SQL queries	Via tool
GZIP	`app_logs.gz`	Compressed log archive	After extract
Text	`transactions.txt`	Plain ledger (new.txt req. step 9)	✅ Yes

Write Strategy

```
Every Product CRUD operation:
  1. Update PRODUCT_CATALOG dict (in-memory, instant)
  2. save_product_db()  → SQLite  (durable)
  3. save_products()    → JSON    (human-readable backup)

main.py also writes:
  4. save_to_xml()      → XML
  5. save_to_pickle()   → Pickle
  6. save_transaction() → transactions.txt
  7. Logger archives    → app_logs.gz
```

Why is data saved TWICE (DB + JSON)?

Reason	Explanation
Redundancy	If one format corrupts, the other is still readable
Demo purpose	Academic requirement to show multiple persistence formats
Human readability	SQLite needs a viewer; JSON opens in any text editor
Flask API	Reads from SQLite (fast); JSON is a fallback export

---

5. OOP Concepts

5.1 Abstract Base Class (ABC) — `models/product.py`

```
from abc import ABC, abstractmethod

class Product(ABC):
    @abstractmethod
    def display_info(self) -> str: ...   # must be implemented by subclasses

    @abstractmethod
    def to_dict(self) -> dict: ...       # for serialization
```

Concrete subclasses: `Electronics`, `Clothing`, `Grocery`

5.2 Dataclass — `models/user.py`

```
from dataclasses import dataclass, field
from datetime import datetime

@dataclass
class User:
    name: str
    id: int
    email: str
    role: UserRole = UserRole.CUSTOMER
    created_at: str = field(default_factory=lambda: datetime.now().isoformat())
```

`@dataclass` auto-generates `__init__`, `__repr__`, `__eq__` — saves boilerplate.

5.3 Enum — `models/user.py`, `models/order.py`

```
from enum import Enum

class UserRole(Enum):
    CUSTOMER = "Customer"
    ADMIN    = "Admin"
    SELLER   = "Seller"

class OrderStatus(Enum):
    PENDING    = "Pending"
    CONFIRMED  = "Confirmed"
    SHIPPED    = "Shipped"
    DELIVERED  = "Delivered"
    CANCELLED  = "Cancelled"
```

Why Enum? Prevents typo bugs (`"Custmer"` vs `"Customer"`), enables IDE autocomplete, makes code self-documenting.

5.4 Encapsulation

```
class Product(ABC):
    def __init__(self, product_id, name, price, stock):
        self._product_id = product_id   # protected
        self._name       = name
        self.__price     = price        # private (name-mangled)
        self._stock      = stock

    @property
    def price(self):          # controlled read access
        return self.__price

    def update_stock(self, delta: int):  # controlled write access
        self._stock += delta
```

5.5 Polymorphism

```
products = [Electronics(...), Clothing(...), Grocery(...)]
for p in products:
    print(p.display_info())   # calls DIFFERENT implementation for each type
```

---

6. API Endpoints

Products

Method	Endpoint	Description	Body / Query
`GET`	`/api/products`	List all products	`?category=Electronics&in_stock=true`
`GET`	`/api/products/<id>`	Single product	—
`POST`	`/api/products`	Create product	`{type, product_id, name, price, stock}`
`PATCH`	`/api/products/<id>/stock`	Update stock	`{stock: 50}`
`DELETE`	`/api/products/<id>`	Delete product	—

Users

Method	Endpoint	Description
`GET`	`/api/users`	All users
`GET`	`/api/users/<id>`	Single user
`POST`	`/api/users`	Create user
`GET`	`/api/users/next-id`	Auto-generate next ID

Orders

Method	Endpoint	Description
`GET`	`/api/orders`	All orders (`?status=Confirmed`)
`GET`	`/api/orders/<id>`	Single order
`POST`	`/api/orders`	Place new order
`PATCH`	`/api/orders/<id>/status`	Update status

System

Method	Endpoint	Description
`GET`	`/api/stats`	Dashboard statistics
`GET`	`/api/logs?n=100`	Last N log lines
`GET`	`/api/datafiles`	All data files with format/size
`GET`	`/api/health`	Health check

---

7. Requirements Coverage (new.txt)

Step	Requirement	Where Implemented
1	Products (3 types)	`models/product.py` + `patterns/factory.py`
2	Users with roles	`models/user.py` (dataclass + enum)
3	Orders	`models/order.py`
4	Payment strategies	`patterns/strategy.py`
5	Factory pattern	`patterns/factory.py`
6	Singleton Logger	`patterns/singleton.py`
7	Persistence (JSON/XML/Pickle)	`persistence/` folder
8	SQLite + GZIP	`persistence/db_handler.py`, `gzip_handler.py`
9	`transactions.txt`	`utils/helpers.py → save_transaction()`
10	`assert` + `pdb`	`utils/helpers.py → transfer(), calculate_discount()`
11	`tabulate` report	`main.py` Step 12 (grid-format product summary)
12	Flask REST API	`app.py` (14 endpoints)
13	Frontend Dashboard	`templates/index.html` + `static/js/main.js`
